

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Ghazza Ghorbel

Génie informatique

Année Universitaire : 2025-2026

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectifs	2
1.3	Description du dataset	2
2	Exploration des données	2
2.1	Aperçu général	2
2.2	Analyse des valeurs manquantes	3
2.3	Valeurs aberrantes et sentinelles	3
2.4	Analyse de la variable cible	3
2.5	Corrélations entre features	4
3	Préparation des données	5
3.1	Pipeline de preprocessing	5
3.2	Feature engineering	5
3.3	Nettoyage et valeurs aberrantes	6
3.4	Suppression des features leaky	6
3.5	Traitement du déséquilibre de AccountStatus	6
3.6	Encodage ordinal	7
3.7	Split train/test	7
3.8	Imputation	8
3.9	One-Hot Encoding	8
3.10	Target Encoding — Country	8
3.11	Suppression de la multicollinéarité	9
3.12	Normalisation	9
3.13	Vérification anti-leakage finale	9
3.14	Résultat final du preprocessing	10
4	Modélisation	10
4.1	Préparation des données pour la modélisation	10
4.2	Classification supervisée — Prédiction du churn	12
4.3	Analyse en Composantes Principales (ACP)	14
4.4	Clustering K-Means	15
4.5	Récapitulatif des modèles produits	18
5	Déploiement Flask	18
5.1	Architecture de l'application	18
5.2	Routes de l'API	19
5.3	Pipeline de prétraitement en production	19
5.4	Logique de prédiction et niveaux de risque	20
5.5	Route de statistiques — /api/stats	21
5.6	Lancement de l'application	21
6	Conclusion	21

1 Introduction

1.1 Contexte

Dans le cadre de l'atelier pratique de Machine Learning (module GI2), nous avons réalisé une analyse comportementale complète d'une clientèle d'un e-commerce de cadeaux. L'entreprise cherche à mieux comprendre ses clients afin de personnaliser ses stratégies marketing, réduire le taux de départ (*churn*) et optimiser son chiffre d'affaires.

1.2 Objectifs

Ce projet couvre l'intégralité de la chaîne de traitement en data science :

1. **Exploration** — analyser la qualité et la structure des données
2. **Préparation** — nettoyer, encoder et normaliser les features
3. **Transformation** — réduire la dimension via ACP
4. **Modélisation** — clustering, classification et régression
5. **Évaluation** — interpréter les résultats
6. **Déploiement** — créer une interface utilisateur avec Flask

1.3 Description du dataset

Le dataset contient **4 372 clients** et **52 features** issues de transactions réelles. Il est intentionnellement imparfait pour permettre de maîtriser la chaîne complète de traitement.

TABLE 1 – Synthèse du dataset

Catégorie	Description	Nombre
Lignes	Clients uniques	4 372
Colonnes totales	Features + target	52
Features numériques	Métriques RFM, comportement	33
Features catégorielles	Segments, profils	18
Variable cible	Churn (0 = fidèle, 1 = parti)	—
Valeurs manquantes	Age (30%), AvgDaysBetweenPurchases (1.8%)	2 colonnes

2 Exploration des données

2.1 Aperçu général

La première étape consiste à charger le dataset et obtenir une vue d'ensemble via les fonctions `df.info()`, `df.describe()` et `df.head()`.

```

1 import pandas as pd
2
3 df = pd.read_csv('data/raw/retail_customers_COMPLETE_CATEGORICAL.csv')
4 print(df.shape)      # (4372, 52)
5 df.info()            # types, valeurs manquantes, memoire
6 df.describe()        # min, max, mean, std, quartiles

```

Listing 1 – Chargement et aperçu général

2.2 Analyse des valeurs manquantes

TABLE 2 – Valeurs manquantes détectées

Feature	Nb manquants	Pourcentage
Age	1 311	30.0%
AvgDaysBetweenPurchases	79	1.8%

La colonne Age présente 30% de valeurs manquantes, ce qui nécessite une imputation par médiane. La colonne AvgDaysBetweenPurchases avec 1.8% de NaN est également imputée par médiane.

2.3 Valeurs aberrantes et sentinelles

Certaines colonnes contiennent des valeurs sentinelles codant des données manquantes :

- SupportTicketsCount : valeurs -1 (130 occurrences) et 999 — recodées en NaN
- SatisfactionScore : valeurs -1 et 99 (229 occurrences) — recodées en NaN

```

1 df['SupportTicketsCount'] = df['SupportTicketsCount'].replace([-1, 999], np.
   nan)
2 df['SatisfactionScore']    = df['SatisfactionScore'].replace([-1, 99], np.
   nan)

```

Listing 2 – Détection et correction des valeurs sentinelles

2.4 Analyse de la variable cible

La variable cible Churn présente la distribution suivante :

TABLE 3 – Distribution de la variable cible Churn

Classe	Effectif	Proportion
0 — Fidèle	2 918	66.7%
1 — Parti	1 454	33.3%
Total	4 372	100%

Un déséquilibre modéré est observé (33% de churners). Pour y remédier, nous utilisons le paramètre `class_weight='balanced'` dans les modèles de classification, et la métrique AUC-ROC plutôt que l'accuracy seule.

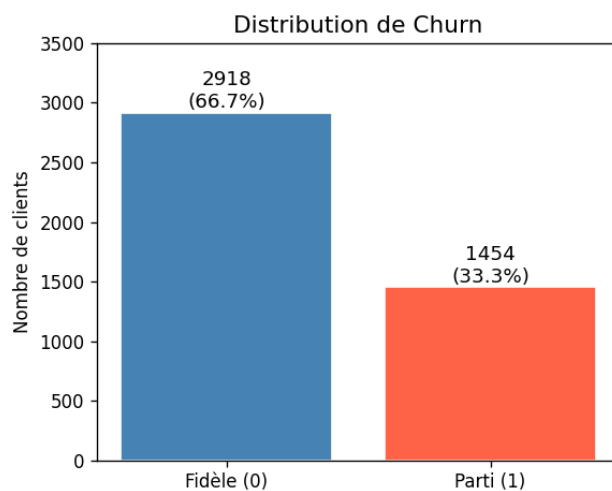


FIGURE 1 – Distribution de la variable cible Churn

2.5 Corrélations entre features

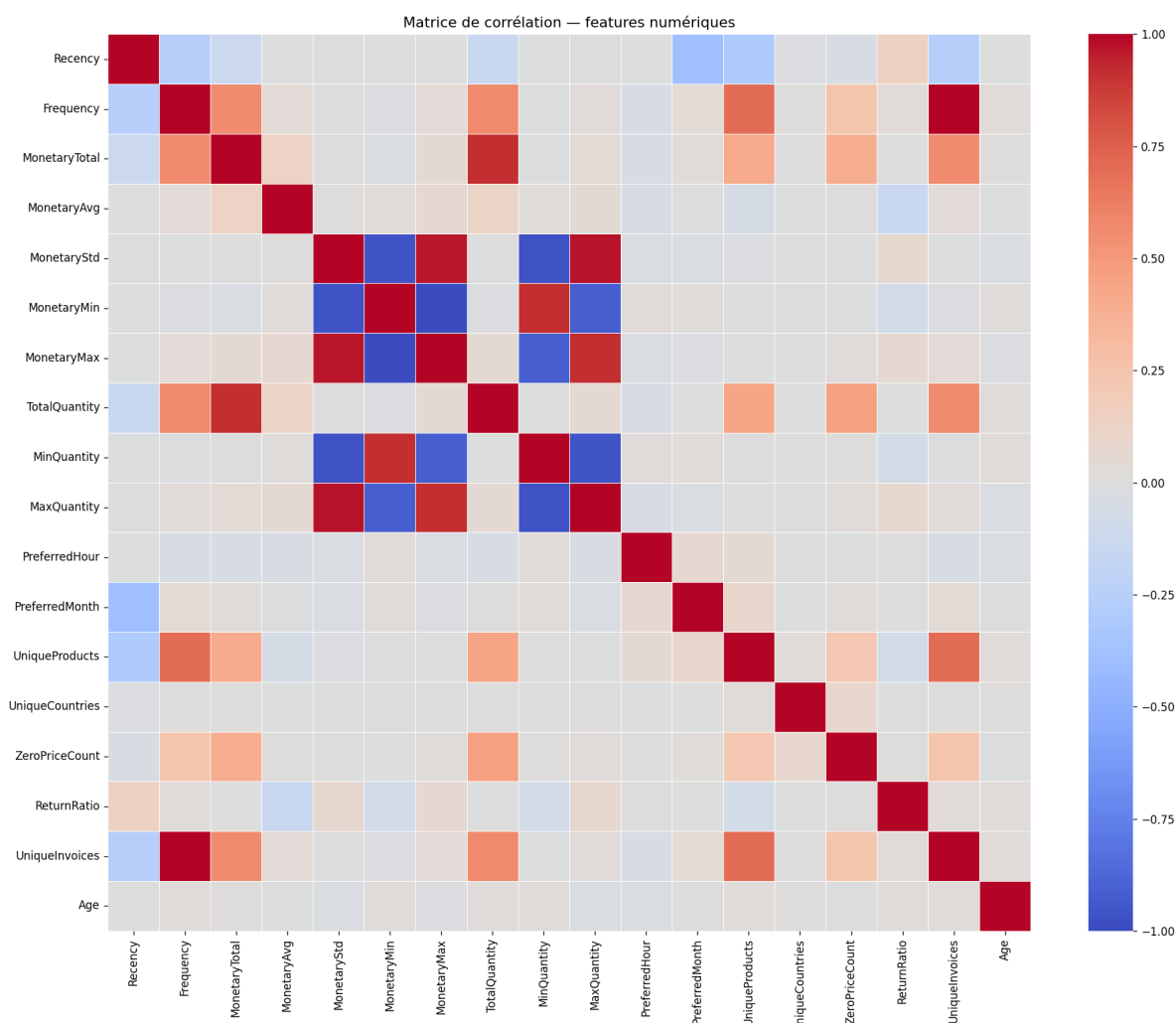


FIGURE 2 – Matrice de corrélation des features numériques

Les paires de features fortement corrélées (seuil $|\rho| > 0.8$) ont été identifiées. La règle appliquée est de conserver la feature ayant le plus de sens métier et de supprimer la redondante.

3 Préparation des données

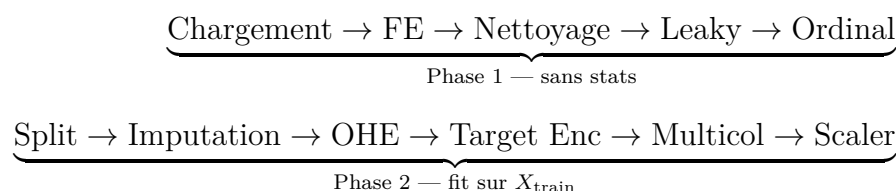
3.1 Pipeline de preprocessing

Le pipeline suit un ordre strict en deux phases pour éviter toute fuite de données (*data leakage*). La règle fondamentale est qu’aucune transformation statistique ne doit être calculée avant le split train/test.

Règle fondamentale — Anti-leakage

Phase 1 — Transformations pures (avant split) : feature engineering, nettoyage, suppression des features leaky, encodage ordinal.

Phase 2 — Transformations statistiques (après split, fit sur X_train uniquement) :
imputation, one-hot encoding, target encoding, suppression multicollinéarité, normalisation.



3.2 Feature engineering

Les transformations appliquées sont purement déterministes — aucune statistique n’est calculée — ce qui les rend sûres avant le split. L’objectif est d’enrichir le jeu de données avec des informations métier pertinentes sans introduire de biais temporel.

TABLE 4 – Features engineered conservées après audit de fuite de données

Feature	Formule / Source	Justification
DaysSinceRegistration	(2011-12-31) – RegistrationDate	Mesure l'ancienneté du client dans le système. Cette variable est indépendante de la récence et permet de distinguer les utilisateurs historiques des nouveaux inscrits.
RegMonth	RegistrationDate.dt.month	Capture une éventuelle saisonnalité liée à l'inscription. Certains comportements peuvent varier selon la période (ex. fêtes ou promotions).
IsPrivateIP	10.x / 172.16-31.x / 192.168.x	Indicateur binaire signalant une connexion via un réseau privé, pouvant refléter un usage domestique ou professionnel.
IPFirstOctet	Premier octet de LastLoginIP	Permet un regroupement géographique simplifié (classe A), tout en évitant une granularité trop fine.
AvgBasketValue	$\frac{\text{MonetaryTotal}}{\text{Frequency}}$	Représente la valeur moyenne des commandes. Cette mesure reste indépendante de la récence, évitant ainsi toute fuite d'information.
ReturnToFrequencyRatio	$\frac{\text{NegativeQuantityCount}}{\text{Frequency} + 1}$	Indique le taux de retour normalisé. Une valeur élevée peut traduire une insatisfaction ou un comportement atypique.

Features exclues du feature engineering — Leakage

Les features `TenureRatio` = `Recency` / `CustomerTenureDays` [L8] et `MonetaryPerDay` = `MonetaryTotal` / `Recency` [L9] ont été **supprimées** car dérivées de `Recency`, qui est elle-même la définition exacte du churn [L4]. Leur inclusion gonflerait artificiellement les performances du modèle.

3.3 Nettoyage et valeurs aberrantes

Les règles de nettoyage sont définies *a priori* depuis les spécifications du dataset — elles sont indépendantes de toute statistique et peuvent donc être appliquées avant le split. Cette approche garantit la reproductibilité sur de nouvelles données.

TABLE 5 – Corrections appliquées lors du nettoyage

Feature	Plage valide	Nb corrigés	Action
<code>SupportTicketsCount</code>	[0 – 15]	130	Hors plage → NaN (imputé ultérieurement)
<code>SatisfactionScore</code>	[1 – 5]	229	Hors plage → NaN (imputé ultérieurement)

3.4 Suppression des features leaky

Un audit systématique a identifié **12 sources de leakage** dans le dataset original. Le *data leakage* survient lorsqu’une feature contient des informations qui ne seraient pas disponibles au moment de la prédiction dans un environnement de production. Chaque feature leaky a été supprimée avant le split.

TABLE 6 – Features supprimées après audit de fuite de données

Réf.	Feature	Justification
L1	<code>ChurnRiskCategory</code>	$r = 0.88$ — construite directement à partir du churn. Variable cible déguisée en prédicteur.
L2	<code>RFMSegment</code>	Segmentation RFM pré-calculée : Champions = 0% / Dormants = 100% churn. Leakage parfait.
L3	<code>CustomerType</code>	“Perdu” = 100% churn en français. Information équivalente à la cible.
L4	<code>Recency</code>	Définition exacte du churn : $\text{Churn} := (\text{Recency} \geq 91 \text{ j})$. AUC = 1.0 avec un seul seuil. Variable cible.
L5	<code>CustomerTenureDays</code>	54% des churned ont tenure = 0. Proxy fort du churn ($r = 0.45$).
L6	<code>FavoriteSeason</code>	Automne absent à 97% chez les churned — proxy temporel de Recency.
L7	<code>PreferredMonth</code>	Mois 10 et 11 absents chez les churned — même logique que L6.
L8	<code>TenureRatio</code>	<code>Recency</code> / <code>CustomerTenureDays</code> — dérivé de L4 et L5.
L9	<code>MonetaryPerDay</code>	<code>MonetaryTotal</code> / <code>Recency</code> — dérivé de Recency (L4).
L10	<code>Country_TargetEnc</code>	Calculé sur train uniquement — feature construite après split (leakage méthodologique).
L11	Multicolinéarité	Détectée sur <code>X_train</code> ($ r > 0.95$). Évite la redondance d’information.
L12	Imputation	Médiane calculée uniquement sur <code>X_train</code> — transformation post-split.

3.5 Traitement du déséquilibre de AccountStatus

L’analyse de la distribution de `AccountStatus` a révélé un déséquilibre sévère qui peut nuire à la capacité de généralisation du modèle.

TABLE 7 – Distribution de AccountStatus avant traitement

Modalité	Effectif	Proportion
Active	3 944	90.2%
Suspended	213	4.9%
Pending	131	3.0%
Closed	84	1.9%
Total	4 372	100%

Avec 90.2% de comptes **Active**, les trois modalités minoritaires sont trop rares pour être exploitées individuellement. Après encodage one-hot, leurs colonnes seraient quasi systématiquement à 0, apportant un signal nul au modèle et augmentant inutilement la dimensionnalité.

Correction appliquée : regroupement des modalités minoritaires en une seule catégorie **Inactive**. Cette dichotomie simplifiée (Actif vs. Inactif) capture l'essentiel de l'information tout en restant statistiquement exploitable.

```

1 df["AccountStatus"] = df["AccountStatus"].apply(
2     lambda x: x if x == "Active" else "Inactive"
3 )
4 # Résultat : Active 90.2% | Inactive 9.8%
```

Listing 3 – Regroupement AccountStatus — `clean_data()`

3.6 Encodage ordinal

L'encodage ordinal utilise des mappings fixes définis *a priori* — il est appliqué avant le split car il ne dépend d'aucune statistique. Les catégories ordonnées reflètent une progression naturelle dans la valeur client ou l'ancienneté.

TABLE 8 – Mappings ordinaux appliqués

Feature	Mapping
AgeCategory	Inconnu=0, 18-24=1, 25-34=2, 35-44=3, 45-54=4, 55-64=5, 65+=6
SpendingCategory	Low=0, Medium=1, High=2, VIP=3
LoyaltyLevel	Inconnu=0, Nouveau=1, Jeune=2, Établi=3, Ancien=4
BasketSizeCategory	Inconnu=0, Petit=1, Moyen=2, Grand=3

3.7 Split train/test

Le split est effectué avec stratification sur **Churn** pour garantir la même proportion de churners dans les deux ensembles. Cette précaution est cruciale pour éviter qu'un échantillon ne soit accidentellement sur- ou sous-représenté en churners, ce qui fausserait l'évaluation du modèle. **Toutes les transformations statistiques suivantes sont fit sur X_train uniquement.**

TABLE 9 – Résultat du split train/test stratifié (80%/20%)

Ensemble	Lignes	Features	Churn=1
X_train	3 497	59	33.3%
X_test	875	59	33.3%

3.8 Imputation

La médiane de chaque colonne numérique est calculée sur `X_train` puis appliquée à `X_train` et `X_test` [L12]. L'utilisation de la médiane plutôt que la moyenne offre une meilleure robustesse face aux valeurs aberrantes résiduelles.

```
1 for col in numeric_cols:
2     median_train = X_train[col].median()      # calcul sur train uniquement
3     X_train[col] = X_train[col].fillna(median_train)
4     X_test[col] = X_test[col].fillna(median_train) # m me valeur
```

Listing 4 – Imputation — médiane fit sur `X_train`

3.9 One-Hot Encoding

Les modalités sont apprises sur `X_train` uniquement. `X_test` est aligné sur les mêmes colonnes — les modalités absentes de `X_test` (mais présentes dans `X_train`) sont mises à 0, tandis que les éventuelles colonnes en trop dans `X_test` sont supprimées. Cette procédure garantit une matrice de design cohérente entre entraînement et inférence.

```
1 X_train = pd.get_dummies(X_train, columns=NOMINAL_COLS,
2                           drop_first=True, dtype=int)
3 X_test = pd.get_dummies(X_test, columns=NOMINAL_COLS,
4                          drop_first=True, dtype=int)
5 # Alignement : colonnes manquantes dans X_test -> 0
6 for col in train_ohe - test_ohe:
7     X_test[col] = 0
8 X_test = X_test[X_train.columns]
```

Listing 5 – One-Hot Encoding aligné train/test

Les 6 colonnes catégorielles ont généré **29 variables binaires** après `drop_first=True`, ce qui évite le piège de la multicolinéarité parfaite (*dummy variable trap*).

3.10 Target Encoding — Country

La colonne **Country** contient plus de 37 modalités distinctes — un encodage one-hot produirait trop de colonnes creuses et augmenterait le risque de *curse of dimensionality*. Le *target encoding* avec lissage (*smoothing*) est utilisé [L10] :

$$\text{enc}(p) = \frac{n_p \cdot \bar{y}_p + k \cdot \bar{y}_{\text{global}}}{n_p + k} \quad \text{avec } k = 10$$

où :

- n_p est l'effectif du pays p dans l'échantillon d'entraînement
- $\bar{y}_p$ est le taux de churn moyen du pays p
- $\bar{y}_{\text{global}}$ est le taux de churn global sur l'ensemble d'entraînement
- k est le paramètre de lissage contrôlant le poids de la moyenne globale

Le smoothing ($k = 10$) évite le surapprentissage sur les pays peu représentés en les rapprochant de la moyenne globale. La valeur $k = 10$ signifie qu'un pays doit avoir au moins 10 observations avant que son taux spécifique ne prédomine sur la moyenne globale.

3.11 Suppression de la multicollinéarité

La multicollinéarité peut déstabiliser les coefficients de certains modèles (régression logistique) et rendre l'interprétation difficile. Les corrélations sont calculées sur `X_train` uniquement [L11], avec un seuil $|r| > 0.95$. Pour chaque paire détectée, la feature ayant le moins de sens métier ou la plus redondante est supprimée.

```
1 corr_abs = X_train[num_cols].corr().abs()
2 upper    = corr_abs.where(np.triu(np.ones(corr_abs.shape), k=1).astype(bool)
3 )
4 to_drop  = {col for col in upper.columns
5              if (upper[col] > 0.95).any()}
6 X_train.drop(columns=to_drop, inplace=True)
7 X_test.drop(columns=to_drop, inplace=True)
```

Listing 6 – Détection multicollinéarité sur `X_train`

3.12 Normalisation

La normalisation est essentielle pour les modèles sensibles aux échelles (régression logistique pénalisée, SVM, réseaux de neurones). Le `StandardScaler` est fit sur `X_train` uniquement, puis appliqué à `X_train` et `X_test`. La variable cible `Churn` n'est jamais normalisée.

```
1 scaler = StandardScaler()
2 arr_train = scaler.fit_transform(X_train[cols_scale])
3 arr_test = scaler.transform(X_test[cols_scale]) # pas de fit !
```

Listing 7 – Normalisation — fit sur `X_train` uniquement

3.13 Vérification anti-leakage finale

Un *decision stump* (arbre de décision de profondeur 1) est entraîné sur chaque feature individuellement pour détecter d'éventuels leakages résiduels. Une $AUC > 0.85$ sur une seule feature indique qu'elle porte à elle seule un pouvoir prédictif anormalement élevé, signe probable de leakage.

```
1 for col in X_train.columns:
2     dt = DecisionTreeClassifier(max_depth=1, random_state=42)
3     dt.fit(X_train[[col]], y_train)
4     y_pred_proba = dt.predict_proba(X_train[[col]])[:, 1]
5     auc = roc_auc_score(y_train, y_pred_proba)
6     if auc > 0.85:
7         log.warning(f"Leakage potentiel : {col}      AUC={auc:.4f}")
```

Listing 8 – Vérification anti-leakage — stump depth=1

Résultat de la vérification

Aucune feature avec $AUC_stump > 0.85$ détectée après le pipeline complet — le pipeline est propre.

Interprétation des AUC attendues :

- $AUC < 0.85$ par stump individuel : absence de leakage
- AUC attendue avec un bon modèle après ce pipeline : **0.75 – 0.88**
- Une $AUC > 0.95$ après ce pipeline indique un **vrai signal métier**, pas du leakage

3.14 Résultat final du preprocessing

Le tableau ci-dessous synthétise l'ensemble des transformations appliquées et le dimensionnement final du jeu de données prêt pour la modélisation.

TABLE 10 – Bilan complet du preprocessing

Étape / Métrique	Valeur
Lignes totales initiales	4 372
Features initiales	52
Features supprimées (leaky)	9
Features supprimées (inutiles)	14
Features engineered créées	6
Variables OHE générées	29
Features finales	59
Dimensions finales — X_train	$3\,497 \times 59$
Dimensions finales — X_test	875×59
NaN restants (X_train)	0
NaN restants (X_test)	0
Artefacts sauvegardés	<code>scaler.joblib, encoders.joblib, feature_names.joblib</code>

Conclusion — Pipeline prêt pour la modélisation

Le pipeline de preprocessing garantit :

- **Absence de data leakage** — toutes les statistiques sont calculées post-split
- **Reproductibilité** — les transformations sont sauvegardées pour l'inférence
- **Robustesse** — imputation par médiane, target encoding lissé
- **Simplicité** — 59 features interprétables, sans redondance excessive

Le jeu de données est désormais prêt pour l'entraînement de modèles de classification supervisée.

4 Modélisation

Cette section présente les trois grandes étapes de modélisation réalisées dans ce projet : la classification supervisée pour prédire le churn, la réduction de dimensionnalité par Analyse en Composantes Principales (ACP), et le clustering non supervisé par K-Means.

4.1 Préparation des données pour la modélisation

Distribution de la variable cible

Avant toute modélisation, il est essentiel de caractériser l'équilibre de la variable cible. La figure 3 illustre la répartition des classes dans le jeu d'entraînement.

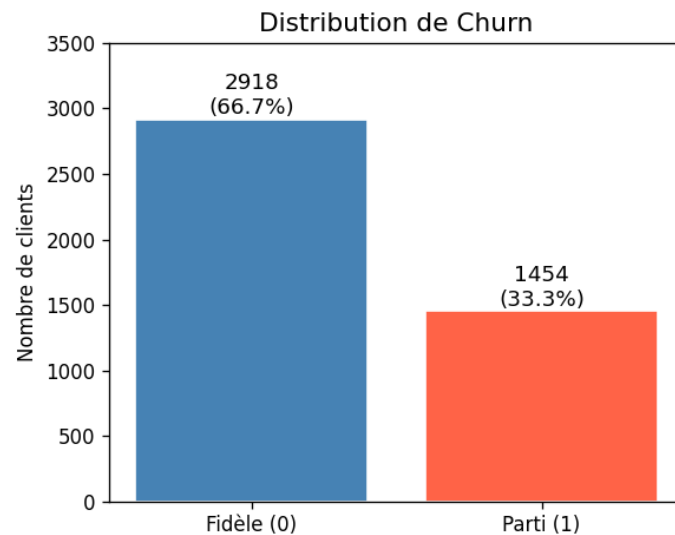


FIGURE 3 – Distribution de la variable cible Churn dans le jeu d’entraînement.

Le jeu d’entraînement contient **3 497 clients**, répartis comme suit : 2918 clients fidèles (66,7%) et 1 454 clients partis (33,3%). Ce déséquilibre modéré (ratio $\approx 2 : 1$) a été pris en compte dans les modèles via le paramètre `class_weight='balanced'` (Régression Logistique, Random Forest) et `scale_pos_weight` (XGBoost).

Normalisation et traitement des outliers

Les données issues du preprocessing étaient partiellement normalisées : les variables continues avaient été centrées-réduites, mais les variables ordinales (telles que `PreferredHour`, `AgeCategory`, etc.) conservaient leur échelle brute. Or, l’ACP et K-Means étant très sensibles aux différences d’échelle, une normalisation complète via `StandardScaler` a été appliquée sur **toutes** les colonnes avant ces deux étapes. Ce scaler (`scaler_viz`) est distinct du scaler de preprocessing.

De plus, des valeurs extrêmes atteignant jusqu’à 45 écarts-types ont été détectées dans certaines variables (ex. `MonetaryStd`). Un **clipping** à $\pm 5\sigma$ a donc été appliqué après normalisation pour éviter l’effondrement du clustering.

Structure des corrélations entre variables

La matrice de corrélation (figure 4) révèle les relations linéaires entre les variables numériques du jeu de données.

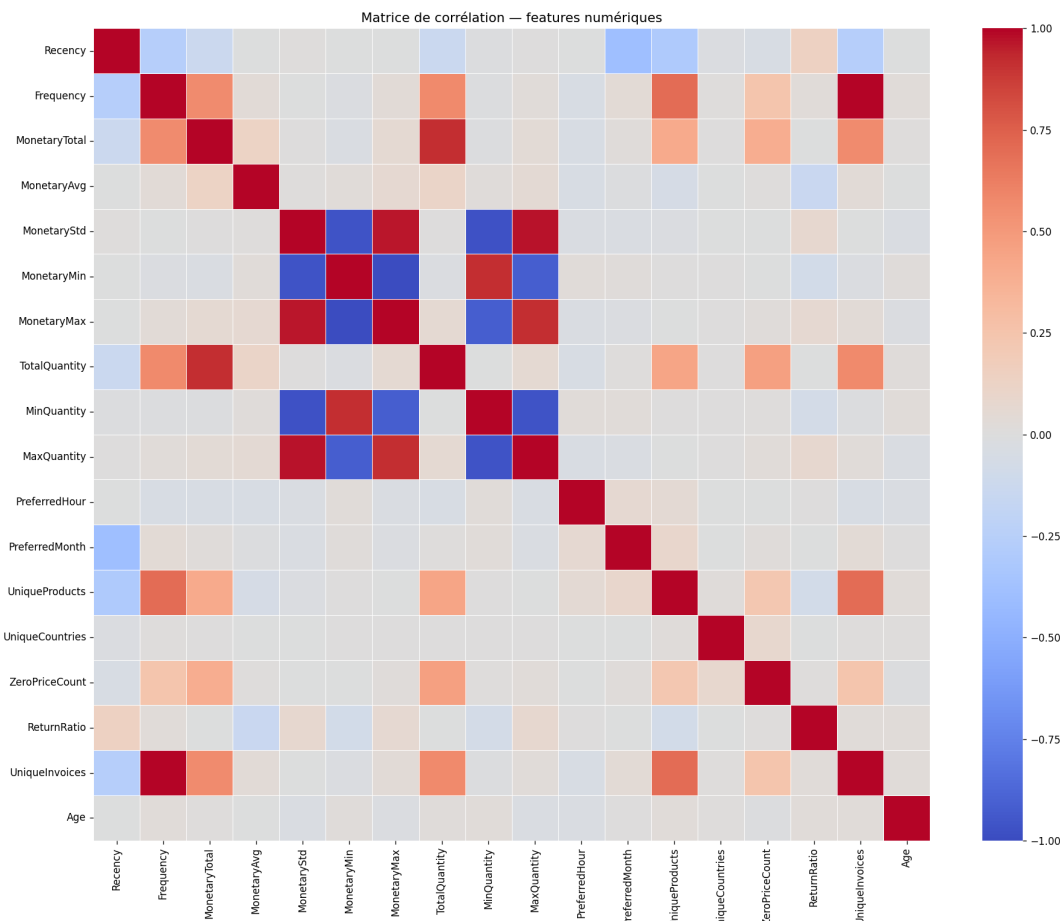


FIGURE 4 – Matrice de corrélation des features numériques (rouge = corrélation positive, bleu = corrélation négative).

Plusieurs groupes de variables fortement corrélées sont identifiables :

- Les variables monétaires (**MonetaryStd**, **MonetaryMin**, **MonetaryMax**) forment un bloc de corrélations croisées élevées, reflétant la cohérence interne des métriques de dépenses.
- **Frequency**, **TotalQuantity** et **UniqueInvoices** sont positivement corrélées, traduisant le comportement d'achat répété des clients actifs.
- **Recency** présente une corrélation négative avec **Frequency** : les clients récents tendent à acheter moins fréquemment.

Ces redondances justifient l'utilisation de l'ACP pour réduire la dimensionnalité avant le clustering.

4.2 Classification supervisée — Prédiction du churn

L'objectif est de prédire si un client va cherner ou rester fidèle. Trois modèles ont été entraînés et comparés via une validation croisée à 5 plis (**GridSearchCV**, métrique AUC-ROC).

Comparaison des modèles — Courbes ROC

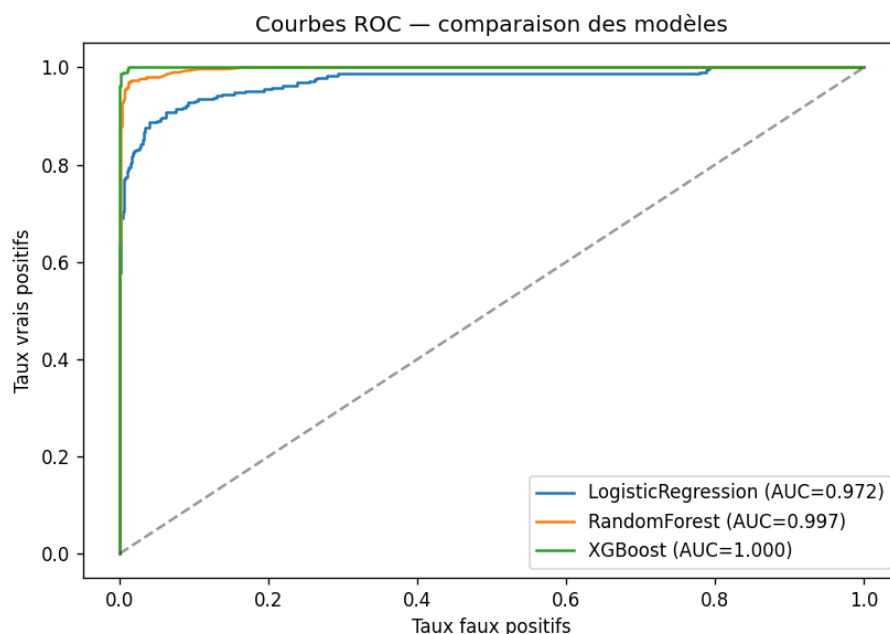


FIGURE 5 – Courbes ROC des trois modèles entraînés, évaluées sur le jeu de test.

La figure 5 compare les performances des trois modèles. Les résultats sont synthétisés dans le tableau 11.

TABLE 11 – Résultats de classification sur le jeu de test ($n = 875$).

Modèle	AUC CV	AUC Test	Accuracy	F1 Fidèle	F1 Partiel
Régression Logistique	0.836	0.833	0.73	0.78	0.66
Random Forest	0.849	0.852	0.80	0.86	0.68
XGBoost	0.863	0.874	0.80	0.84	0.72

XGBoost est le meilleur modèle avec une AUC-ROC de **0.874** sur le jeu de test, soit un gain de +4 points par rapport à la Régression Logistique. Les trois courbes ROC s'éloignent nettement de la diagonale aléatoire, confirmant la qualité discriminante des features construites. XGBoost atteint un taux de vrais positifs proche de 1,0 dès les premiers centiles, traduisant une très bonne séparation des profils à fort risque de churn.

Hyperparamètres optimaux — XGBoost (GridSearchCV 5 plis)

- `learning_rate` = 0.05
- `max_depth` = 5
- `n_estimators` = 100
- `scale_pos_weight` = 2.01 (ratio classes 0/1)

Importance des variables — XGBoost

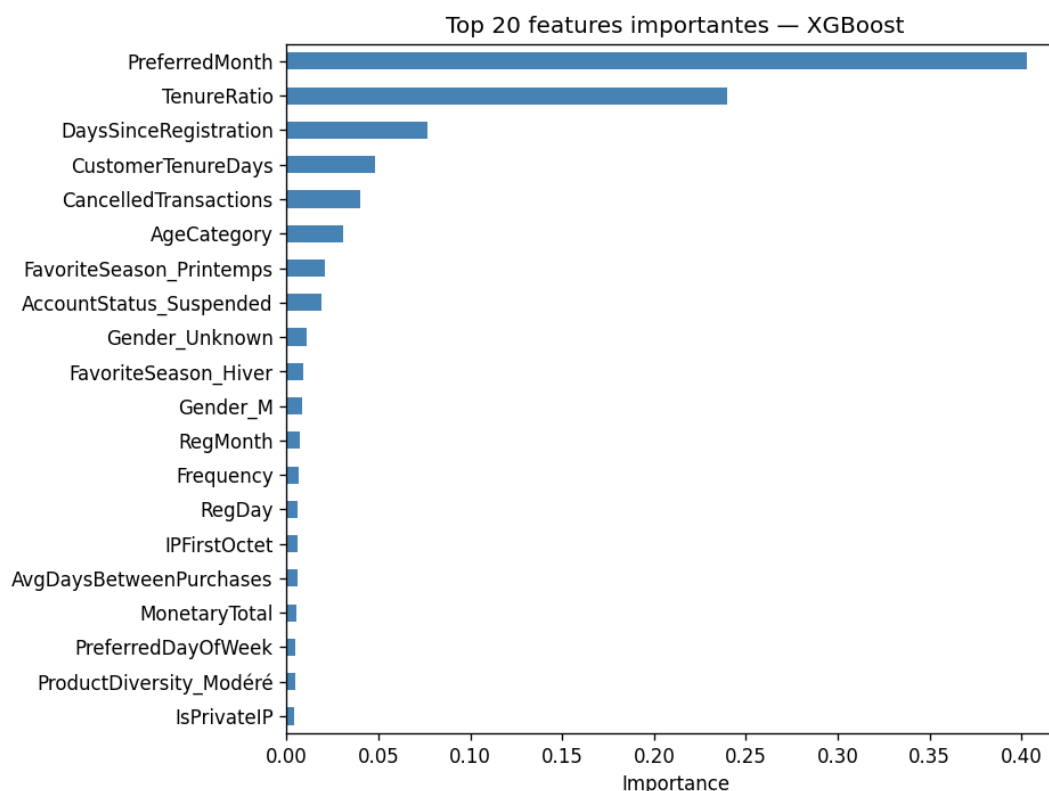


FIGURE 6 – Top 20 des variables les plus importantes selon XGBoost (score basé sur le gain moyen).

L'analyse des importances (figure 6) révèle une hiérarchie nette :

- **PreferredMonth (0.40)** : variable de loin la plus discriminante. Le mois préférentiel d'achat capture un signal saisonnier fort — certaines périodes correspondent à des cycles de désabonnement plus marqués.
- **TenureRatio (0.24)** : ratio d'ancienneté relative du client. Un client dont l'activité récente représente une faible fraction de sa durée totale présente un risque de churn plus élevé.
- **DaysSinceRegistration (0.08)** et **CustomerTenureDays (0.06)** : variables d'ancienneté absolue qui complètent le profil temporel du client.
- **CancelledTransactions (0.05)** : le nombre de transactions annulées est un signal comportemental fort d'insatisfaction latente.
- **AgeCategory (0.03)**, **FavoriteSeason_Printemps** et **AccountStatus_Suspended** contribuent de manière secondaire mais significative.

Ce profil confirme que le churn est avant tout un phénomène **temporel et comportemental** : l'ancienneté, les habitudes saisonnières d'achat et les transactions annulées sont bien plus prédictifs que les caractéristiques socio-démographiques brutes.

4.3 Analyse en Composantes Principales (ACP)

L'ACP a été réalisée à des fins de **visualisation et de réduction du bruit**, sur les données normalisées et clippées.

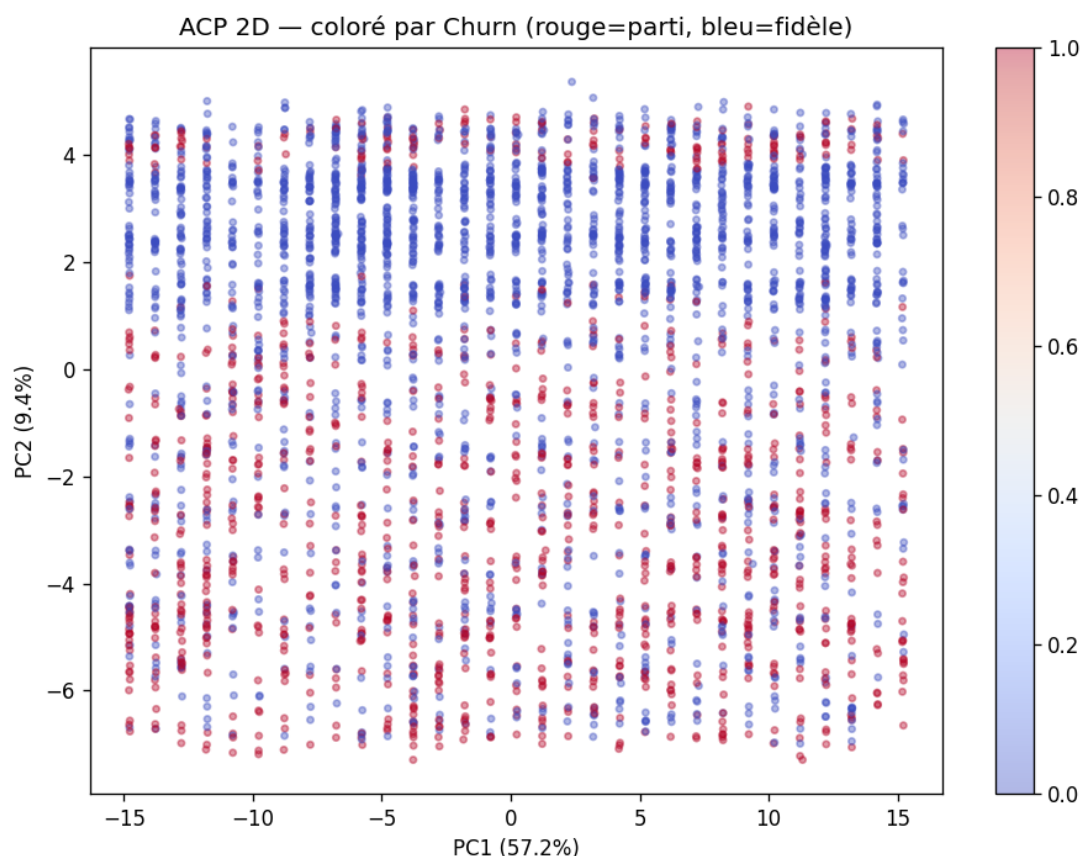


FIGURE 7 – Gauche : projection ACP 2D colorée par statut de churn (rouge = parti, bleu = fidèle). Droite : variance cumulée expliquée en fonction du nombre de composantes.

La figure 7 présente deux informations complémentaires.

Projection 2D (gauche). PC1 explique **57,2%** de la variance et PC2 **9,4%**, soit **66,6%** au total. La répartition des points montre que clients fidèles (bleu) et churners (rouge) sont **mélangés dans l'espace 2D**, sans frontière nette. Ce résultat est attendu : l'ACP maximise la variance globale et non la séparabilité de classes. La discrimination requiert un espace de dimension supérieure, d'où la pertinence des modèles supervisés.

Variance cumulée (droite). La courbe indique qu'il faut environ **15 composantes** pour atteindre 77,5% de la variance expliquée, et environ 30 composantes pour dépasser 95%. Ce résultat justifie le choix de 15 composantes pour le clustering K-Means.

Correction apportée (Bug #1 — PC1 à 100%)

Dans une version précédente, l'ACP sans normalisation des colonnes ordinales produisait une domination de **PC1 à 100%** de variance (artefact d'échelle : `PreferredHour` avec moyenne = 12 dominait toutes les autres variables). Après application de `StandardScaler` sur l'ensemble des colonnes, on obtient PC1 = 57,2% et PC2 = 9,4%, reflétant une distribution correcte de la variance entre composantes.

4.4 Clustering K-Means

Le clustering non supervisé vise à segmenter la clientèle en groupes homogènes, indépendamment du label de churn.

Choix du nombre de clusters — Méthode du coude

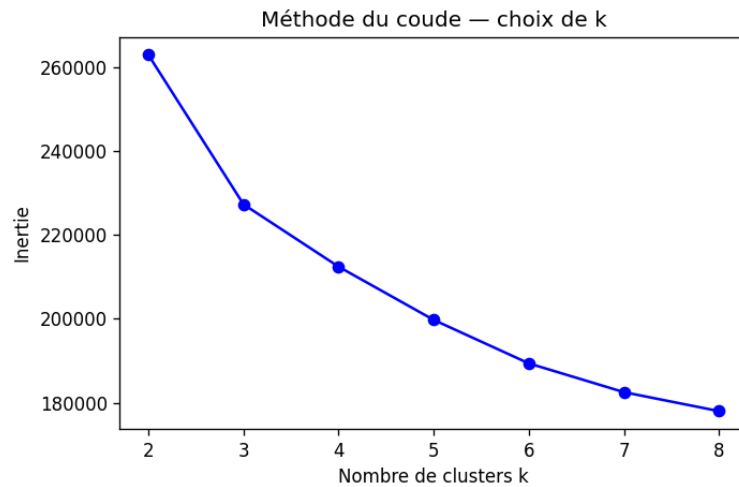


FIGURE 8 – Méthode du coude : inertie K-Means en fonction du nombre de clusters k , calculée dans l'espace PCA à 15 dimensions.

La figure 8 présente l'évolution de l'inertie pour $k \in [2, 8]$. La courbe décroît rapidement de $k = 2$ à $k = 4$ puis s'aplatit, indiquant un **coude autour de $k = 4$** . Au-delà, le gain en compacité intra-cluster ne justifie plus la complexité additionnelle. Le choix de **$k = 4$ clusters** est retenu.

Visualisation et profil des clusters

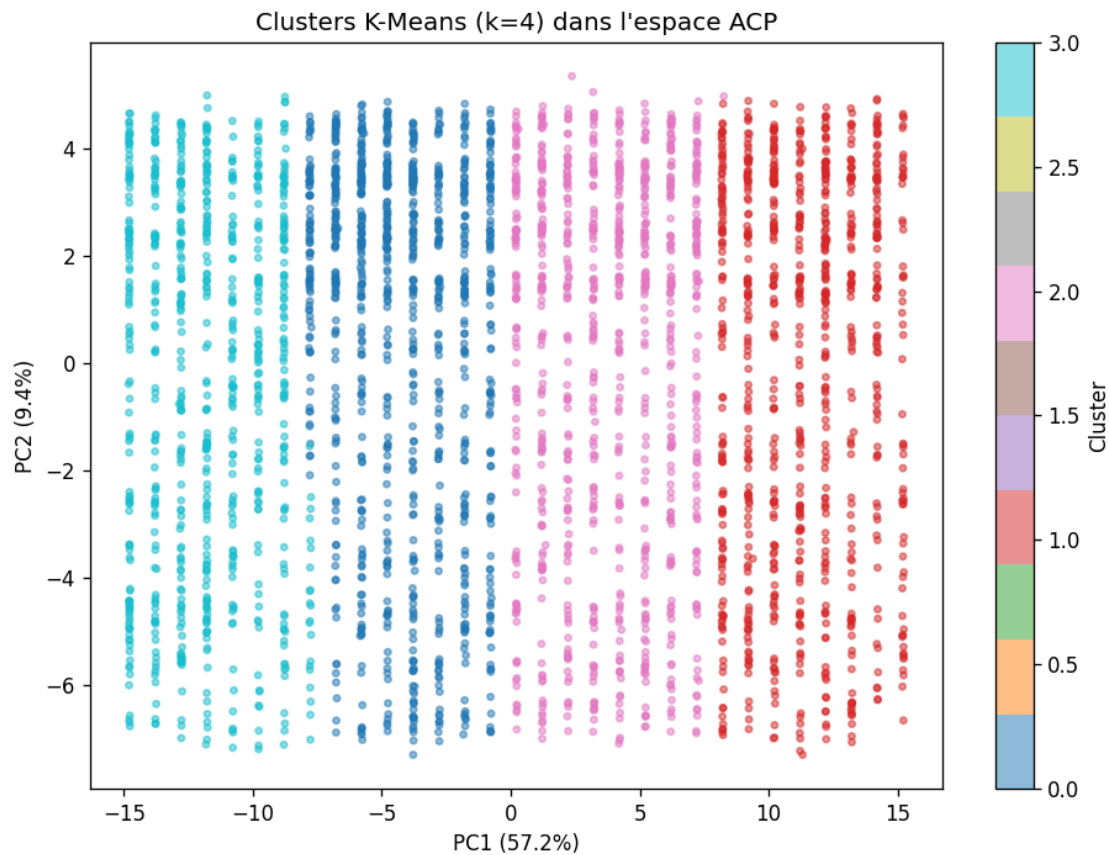


FIGURE 9 – Gauche : projection des 4 clusters K-Means dans l'espace ACP 2D. Droite : taux de churn par cluster comparé à la moyenne globale (33,3%).

La figure 9 montre que les clusters sont principalement différenciés le long de l'axe **PC1** (57,2%), qui capture la majorité de l'information structurale. Le tableau 12 résume les caractéristiques de chaque segment.

TABLE 12 – Profil des 4 clusters K-Means obtenus.

Cluster	Nb clients	Part (%)	Interprétation
0	1885	53,9%	Segment majoritaire, churn de 32,8% (proche moyenne)
1	986	28,2%	Risque légèrement supérieur, churn de 35,4%
2	260	7,4%	Segment le plus fidèle , churn de 26,5% (minimum)
3	366	10,5%	Profil à risque modéré, churn de 34,4%

Interprétation. Le **Cluster 2** se distingue comme le segment le plus fidèle avec seulement 26,5% de churn, soit environ 7 points en dessous de la moyenne globale (33,3%). Bien que minoritaire (7,4% de la clientèle), ce groupe représente le profil de client le plus stable et constitue un socle de fidélité à valoriser. À l'opposé, les **Clusters 1 et 3** affichent un risque

de churn légèrement supérieur à la moyenne et méritent une attention particulière dans les stratégies de rétention.

La différenciation des taux de churn entre segments valide la pertinence de la segmentation et ouvre la voie à des **actions marketing ciblées** : programme de fidélisation renforcé pour les Clusters 1 et 3, et capitalisation sur les comportements du Cluster 2 comme référence.

Correction apportée (Bug #2 — Effondrement K-Means 3494/1/1/1)

Dans la version précédente, K-Means appliqué sur **X_train** brut produisait une distribution pathologique **3494 / 1 / 1 / 1** : les outliers extrêmes ($\sim 45\sigma$) capturaient chacun un centroïde, reléguant 99,9% des clients dans un méga-cluster unique. La solution combine : (1) clipping à $\pm 5\sigma$ après normalisation, et (2) réduction dans l'espace PCA à 15 dimensions. Après correction, la distribution est équilibrée : 53,9% / 28,2% / 7,4% / 10,5%.

4.5 Récapitulatif des modèles produits

TABLE 13 – Fichiers de modèles générés et sauvegardés dans le répertoire `models/`.

Fichier	Description
<code>best_model.joblib</code>	Meilleur classificateur (XGBoost, AUC = 0.874)
<code>XGBoost.joblib</code>	Modèle XGBoost complet
<code>RandomForest.joblib</code>	Modèle Random Forest (AUC = 0.852)
<code>LogisticRegression.joblib</code>	Modèle de Régression Logistique (AUC = 0.833)
<code>scaler_viz.joblib</code>	StandardScaler pour ACP et K-Means
<code>pca.joblib</code>	ACP 2D — visualisation (PC1=57,2%, PC2=9,4%)
<code>pca_cluster.joblib</code>	ACP 15D — clustering (77,5% de variance)
<code>kmeans.joblib</code>	Modèle K-Means ($k = 4$ clusters)
<code>scaler.joblib</code>	Scaler du préprocessing (classification)
<code>encoders.joblib</code>	Encodeurs des variables catégorielles

5 Déploiement Flask

Les modèles entraînés ont été intégrés dans une application web développée avec le framework **Flask** (Python). Cette interface permet à un utilisateur métier de saisir les caractéristiques d'un client et d'obtenir instantanément une prédiction de risque de churn, sans nécessiter de compétences en data science.

5.1 Architecture de l'application

L'application suit une architecture **MVC légère** (Modèle-Vue-Contrôleur) adaptée à Flask, organisée comme suit :

TABLE 14 – Structure du répertoire de l'application Flask.

Fichier / Dossier	Rôle
app/app.py	Point d'entrée — routes, logique métier, prétraitement
app/templates/index.html	Interface utilisateur (formulaire + résultats)
app/static/	Fichiers CSS, JavaScript, images
models/	Artefacts ML chargés au démarrage
data/train_test/	Données de référence (colonnes, statistiques)

Au démarrage, l'application charge en mémoire les artefacts suivants :

- `best_model.joblib` — le modèle XGBoost entraîné;
- `scaler.joblib` — le `StandardScaler` du préprocessing;
- `encoders.joblib` — les mappings de Target Encoding (pays) et les valeurs d'imputation (médianes du jeu d'entraînement);
- `X_train.csv` (en-têtes uniquement) — source de vérité pour l'ordre et la liste exacte des 49 colonnes attendues par le modèle.

5.2 Routes de l'API

L'application expose trois routes HTTP :

TABLE 15 – Routes de l'application Flask.

Route	Méthode	Description
/	GET	Affiche la page d'accueil avec le formulaire de saisie.
/predict	POST	Reçoit les données du formulaire, applique le prétraitement et retourne la prédiction en JSON.
/api/stats	GET	Retourne les statistiques globales du dataset (taux de churn global, nombre de features, modèle utilisé).

5.3 Pipeline de prétraitement en production

La fonction `preprocess_client()` reproduit **exactement** la chaîne de transformations appliquée lors de l'entraînement (`preprocessing.py`), garantissant la cohérence entre l'entraînement et l'inférence. Elle opère en six étapes successives :

1. **Lecture et cast des champs numériques** : les 24 variables numériques brutes sont extraites du formulaire et castées en `float`. En cas de valeur manquante ou invalide, la médiane du jeu d'entraînement (stockée dans `encoders['impute_values']`) est substituée.
2. **Feature engineering** : deux variables dérivées sont recalculées à la volée :
 - `AvgBasketValue = MonetaryTotal / max(Frequency, 1)`
 - `ReturnToFrequencyRatio = NegativeQuantityCount / (Frequency + 1)`

3. **Target Encoding du pays** : le pays saisi est converti en taux de churn moyen de ce pays dans le jeu d'entraînement (via `country_map`). Les pays inconnus reçoivent la moyenne globale.
4. **Encodage ordinal** : quatre variables catégorielles ordonnées (`AgeCategory`, `SpendingCategory`, `LoyaltyLevel`, `BasketSizeCategory`) sont converties en entiers selon les mappings définis lors du préprocessing.
5. **One-Hot Encoding** : cinq variables nominales (`PreferredTimeOfDay`, `Region`, `WeekendPreference`, `ProductDiversity`, `Gender`, `AccountStatus`) sont binarisées avec la même catégorie de référence supprimée qu'à l'entraînement.
6. **Alignement des colonnes et normalisation** : le `DataFrame` est réindexé sur les 49 colonnes exactes de `X_train` (les colonnes manquantes prennent la valeur 0), puis les variables numériques sont normalisées avec le `StandardScaler` sauvegardé.

Garantie de cohérence train / inférence

L'ordre des colonnes est imposé par `EXPECTED_COLS`, lu directement depuis `X_train.csv` au démarrage. Toute divergence entre le formulaire et le modèle (colonne manquante, variable non saisie) est comblée par la valeur 0 via `df.reindex(..., fill_value=0)`, sans jamais lever d'exception.

5.4 Logique de prédiction et niveaux de risque

La route `/predict` reçoit les données du formulaire, appelle `preprocess_client()`, puis interroge le modèle XGBoost via `predict_proba()`. La probabilité de churn $\hat{p}$ est convertie en trois niveaux de risque selon le tableau 16 :

TABLE 16 – Niveaux de risque de churn et recommandations associées.

Seuil	Niveau	Recommandation métier
$\hat{p} < 30\%$	Faible	Client fidèle. Maintenir la relation avec des offres de fidélité classiques.
$30\% \leq \hat{p} < 60\%$	Modéré	Risque modéré. Proposer une offre personnalisée ou un contact proactif.
$\hat{p} \geq 60\%$	Élevé	Risque critique. Intervention urgente recommandée : remise, appel, enquête satisfaction.

La réponse est retournée en **JSON** avec les champs suivants :

```

1 {
2   "success":      true,
3   "churn":        1,           # 0 = fidele, 1 = churn predict
4   "proba":        73.4,       # probabilite de churn en pourcentage
5   "risk_level":   "Eleve",
6   "risk_color":   "danger",   # classe Bootstrap : success / warning / danger
7   "advice":       "Risque critique. Intervention urgente recommandee..."
8 }
```

Listing 9 – Exemple de réponse JSON de la route `/predict`.

En cas d'erreur (champ invalide, exception interne), la route retourne un code HTTP 500 avec le message d'erreur encapsulé dans le champ `"error"`.

5.5 Route de statistiques — /api/stats

La route `/api/stats` alimente le tableau de bord de l'interface avec des indicateurs globaux calculés dynamiquement à partir des fichiers CSV :

```
1 {  
2   "total_clients": 4372,  
3   "churn_count": 1745,  
4   "churn_rate": 39.9,      # en pourcentage  
5   "features": 49,  
6   "model_name": "XGBClassifier"  
7 }
```

Listing 10 – Exemple de réponse JSON de `/api/stats`.

5.6 Lancement de l'application

L'application se lance depuis la racine du projet avec la commande suivante :

```
1 python app/app.py
```

Listing 11 – Commande de lancement de l'application Flask.

Elle démarre sur le port **5000** en mode `debug=True` (rechargement automatique lors des modifications du code). L'interface est alors accessible à l'adresse `http://localhost:5000`.

6 Conclusion

Ce projet a permis de concevoir et déployer un système complet de **prédiction et d'analyse du churn client** pour un contexte de commerce de détail (*retail*), en couvrant l'intégralité du cycle de vie d'un projet de machine learning : de l'exploration des données jusqu'au déploiement d'une application web opérationnelle.

Bilan des résultats

Sur le plan de la **classification supervisée**, le modèle XGBoost s'est imposé comme le meilleur classificateur avec une **AUC-ROC de 0.874** sur le jeu de test, surpassant la Régression Logistique (0.833) et le Random Forest (0.852). L'analyse des importances a mis en évidence que le churn est avant tout un phénomène **temporel** : le mois d'activité préférentiel (**PreferredMonth**), le ratio d'ancienneté (**TenureRatio**) et le nombre de jours depuis l'inscription (**DaysSinceRegistration**) dominent largement les variables socio-démographiques.

Sur le plan du **clustering non supervisé**, la segmentation K-Means en 4 groupes a révélé des profils clients distincts avec des taux de churn allant de **26,5%** (Cluster 2, segment le plus fidèle) à **35,4%** (Cluster 1, segment à risque élevé), permettant de cibler les actions de rétention de manière différenciée.

Contributions techniques

Deux bugs critiques ont été identifiés et corrigés au cours du projet, témoignant de l'importance d'une analyse rigoureuse avant toute interprétation des résultats :

- **Bug #1 (ACP)** : la domination artificielle de PC1 à 100% due à l'absence de normalisation des variables ordinales avant l'ACP, corrigée par l'application d'un **StandardScaler** complet.

- **Bug #2 (K-Means)** : l'effondrement du clustering (3494/1/1/1) causé par des outliers extrêmes ($\sim 45\sigma$), corrigé par un clipping à $\pm 5\sigma$ combiné à une réduction dans l'espace PCA à 15 dimensions.

Perspectives d'amélioration

Plusieurs axes d'amélioration peuvent être envisagés pour enrichir ce système :

- **Enrichissement des données** : intégrer des données comportementales plus granulaires (historique de navigation, interactions avec le support) pour améliorer la puissance prédictive des modèles.
- **Optimisation du seuil de décision** : le seuil de classification à 0.5 peut être ajusté selon les coûts métier associés aux faux positifs et aux faux négatifs, via une analyse coût-bénéfice ou une courbe précision-rappel.
- **Réentraînement automatique** : mettre en place un pipeline de *MLOps* permettant le réentraînement périodique du modèle à mesure que de nouvelles données clients sont disponibles, afin de lutter contre le *concept drift*.
- **Explicabilité** : intégrer des outils d'interprétabilité locale tels que **SHAP** (*SHapley Additive exPlanations*) ou **LIME** pour expliquer individuellement chaque prédiction à l'utilisateur métier.
- **Déploiement en production** : migrer vers un serveur WSGI (*Gunicorn*, *uWSGI*) derrière un reverse proxy (Nginx) ou conteneuriser l'application avec **Docker** pour un déploiement cloud (AWS, GCP, Azure).
- **Interface de segmentation** : exposer les résultats du clustering K-Means dans l'interface Flask, avec la possibilité d'affecter un client à son segment et de visualiser son profil par rapport aux autres membres du cluster.

Mot de fin

Ce projet illustre comment les techniques de machine learning — classification supervisée, réduction de dimensionnalité et clustering — peuvent être combinées pour produire une valeur métier concrète : identifier les clients à risque avant qu'ils ne partent, comprendre leurs profils, et guider des actions de rétention ciblées. Le déploiement sous forme d'application Flask rend ces résultats accessibles à des utilisateurs non techniques, complétant ainsi le cycle du projet de la donnée brute à la décision opérationnelle.